Local PC Performance Diagnoser

> **Collect system metrics from the OS, detect memory problems with code, let AI explain the results.**

A local tool that reads CPU and process data from your PC, stores it as time-series data in SQLite, flags RAM pressure and memory overflow, and uses an LLM to turn the findings into a plain-English diagnosis.

Core pipeline:

```
Collect (CPU + processes) → Save to SQLite → Detect RAM / memory issues → AI explains
```

Covers three layers recruiters care about: **operating systems**, **API + data**, and **LLM**.

Table of Contents

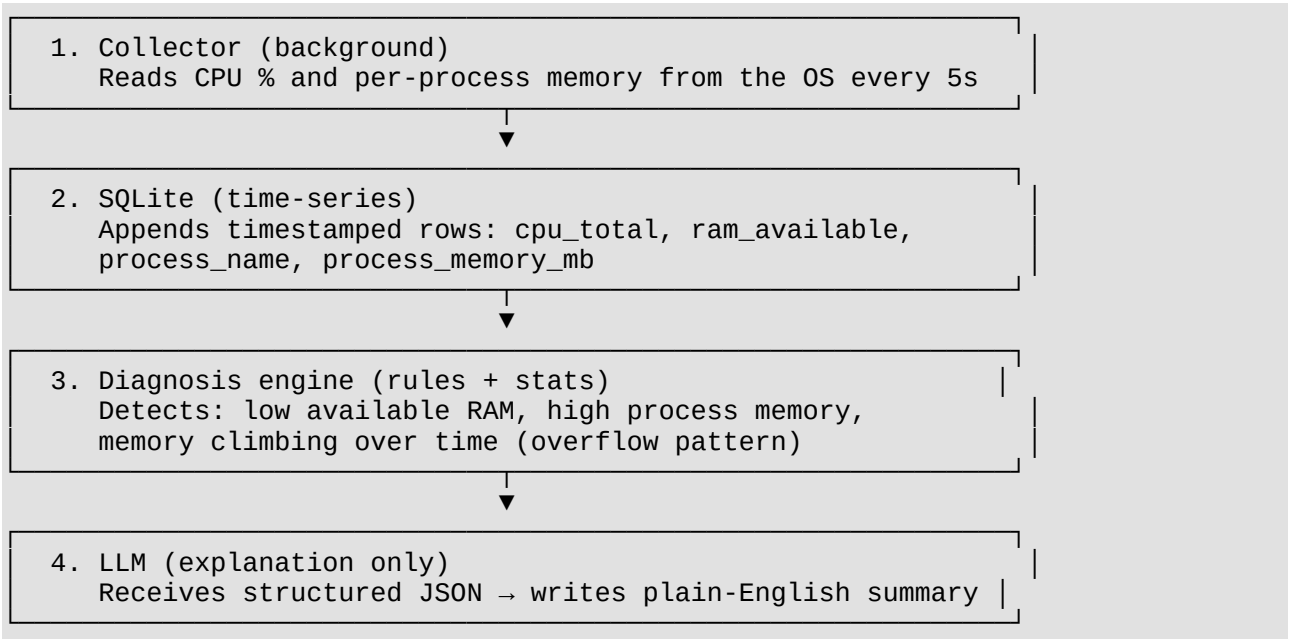
Why This Project

Typical AI chatbot	Local PC Performance Diagnoser
LLM guesses why your PC is slow	OS APIs provide real CPU and memory numbers

Typical AI chatbot	Local PC Performance Diagnoser
No history	Metrics saved over time in SQLite
Generic tips	Diagnosis tied to your actual process list and RAM usage

Small enough to build solo, deep enough to show systems + data + AI skills.

How It Works

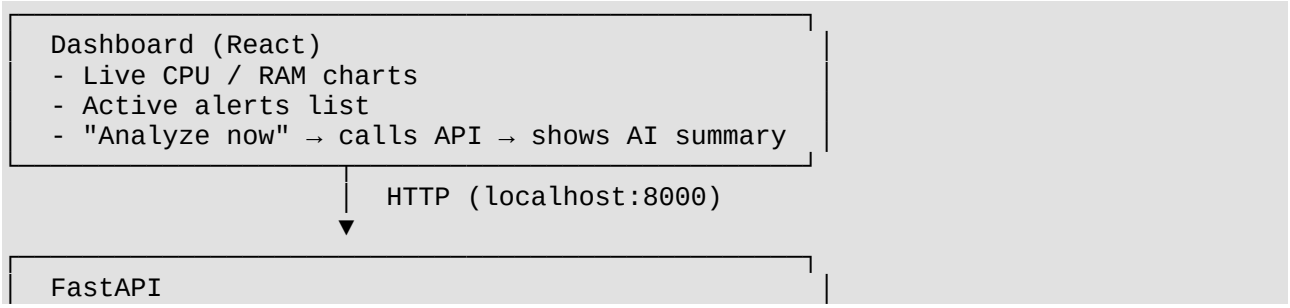


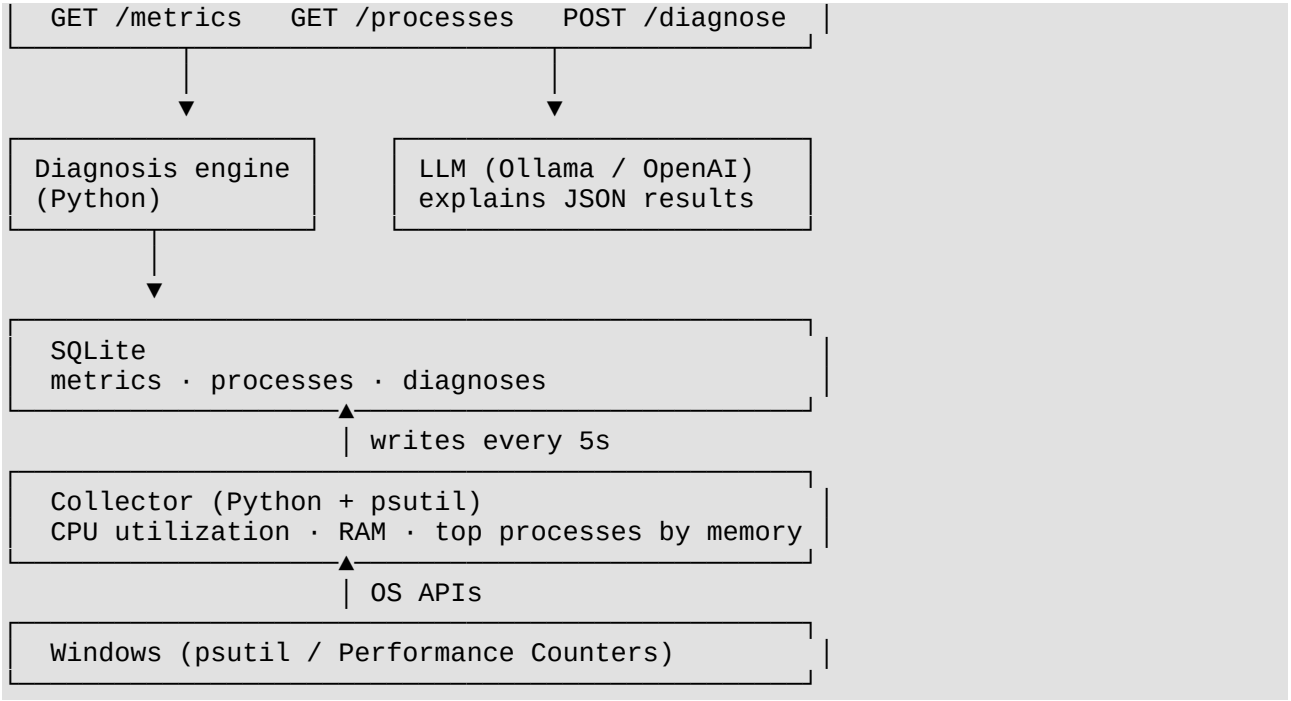
Example: You open a heavy app and the PC stutters.

1. Collector logged RAM dropping from 4 GB → 300 MB over 30 seconds.
2. Engine flagged `chrome.exe` using 2.1 GB and available RAM below 500 MB.
3. LLM output: *"Your PC is low on memory. Chrome is using 2.1 GB — close tabs or quit unused apps to free RAM."*

The LLM did not detect the problem. **Code did.** The LLM only explained it.

Architecture





Data Collection

One background script. No agent involved.

What	How	Interval
CPU usage (total %)	`psutil.cpu_percent()`	Every 5s
Available RAM (MB)	`psutil.virtual_memory().available`	Every 5s
Top 10 processes by memory	`psutil.process_iter()` → name + RSS	Every 5s

Windows note: `psutil` wraps the same OS data Task Manager uses. No admin rights needed for basic metrics.

Storage

Single **SQLite** file: `%LOCALAPPDATA%/PCDiagnoser/data.db`

```
-- System metrics (time-series)
metrics (
  timestamp       INTEGER NOT NULL,   -- Unix ms
  cpu_percent     REAL NOT NULL,
  ram_available_mb REAL NOT NULL,
  ram_used_percent REAL NOT NULL
)

-- Process snapshots (time-series)
process_snapshots (
```

```
  timestamp       INTEGER NOT NULL,
  process_name    TEXT NOT NULL,
  memory_mb       REAL NOT NULL,
  cpu_percent     REAL
)

-- Diagnosis history
diagnoses (
  id              INTEGER PRIMARY KEY,
  timestamp       INTEGER NOT NULL,
  issues          TEXT NOT NULL,      -- JSON: detected problems + evidence
  ai_explanation  TEXT                -- LLM plain-English summary
)
```

Retention: Keep 7 days of raw data, then delete older rows (simple cron or startup cleanup).

Anomaly Detection

Rule-based only for MVP — no ML required.

Issue	Condition	Severity
RAM critical	Available RAM < 500 MB	High
RAM warning	Available RAM < 1 GB OR used > 90%	Medium
Process memory hog	Single process > 1 GB RSS	Medium
Memory overflow pattern	Available RAM dropped > 50% in 60s	High
High CPU	CPU > 90% for 3 consecutive samples (15s)	Medium

Output (JSON passed to LLM):

```
{
  "status": "critical",
  "issues": [
    {
      "type": "ram_critical",
      "message": "Only 280 MB RAM available",
      "evidence": { "ram_available_mb": 280, "ram_used_percent": 96.2 }
    },
    {
      "type": "process_memory_hog",
      "message": "chrome.exe using 2.1 GB",
      "evidence": { "process_name": "chrome.exe", "memory_mb": 2100 }
    }
  ],
  "top_processes": [
    { "name": "chrome.exe", "memory_mb": 2100 },
    { "name": "Code.exe", "memory_mb": 890 }
  ]
}
```

API

FastAPI server on `localhost:8000`.

Endpoint	Method	Description
`/health`	GET	Server + collector status
`/metrics`	GET	Recent CPU/RAM time-series (`?minutes=60`)
`/processes`	GET	Current top processes by memory
`/diagnose`	POST	Run detection engine → return JSON issues
`/analyze`	POST	Run detection → send JSON to LLM → return explanation

Example:

```
curl -X POST http://localhost:8000/analyze

{
  "issues": [ ... ],
  "explanation": "Your PC is running critically low on memory (280 MB free).
Chrome is the largest consumer at 2.1 GB. Close unused browser tabs or quit
Chrome to recover performance."
}
```

Dashboard & AI

Dashboard (React)

- **Live charts:** CPU % and available RAM over the last hour
- **Process table:** Top memory consumers, refreshed every 5s
- **Alerts panel:** Active issues from the detection engine
- **Analyze button:** Calls `POST /analyze` and displays the AI summary

LLM role

- **Input:** Structured JSON from `/diagnose` (never raw SQLite rows)
- **Output:** Short plain-English explanation + 2–3 suggested fixes
- **Default:** Ollama locally; OpenAI optional
- **Rule:** LLM cannot add issues that are not in the JSON

Tech Stack

Layer	Tool	Why
Collector	Python + psutil	Simple, cross-platform, fast to build
Database	SQLite	Zero setup, perfect for local time-series
Detection	Python	Threshold rules, no ML dependency
API	FastAPI	Lightweight REST, auto-generated docs
Dashboard	React + Recharts	Live charts, familiar stack
LLM	Ollama (local)	No API key needed for demo

Project Structure

```
local-pc-performance-diagnoser/
├── README.md
├── collector/
│   ├── main.py              # Background loop: read OS → write SQLite
│   └── db.py                # SQLite helpers
├── engine/
│   ├── detect.py            # RAM / memory overflow rules
│   └── schemas.py           # Pydantic models for diagnosis JSON
├── api/
│   ├── main.py              # FastAPI app
│   └── routes.py            # /metrics, /diagnose, /analyze
├── dashboard/
│   ├── src/
│   │   ├── App.tsx
│   │   ├── MetricsChart.tsx
│   │   └── AnalyzePanel.tsx
│   └── package.json
├── scripts/
│   ├── setup.ps1
│   └── simulate_memory_pressure.py   # Demo: fill RAM for testing
└── requirements.txt
```

Development Plan

Week 1 — Collect & Store

- ☐ SQLite schema + collector loop (CPU, RAM, top 10 processes every 5s)
- ☐ Verify data accumulates correctly over 30+ minutes
- ☐ CLI: `python collector/main.py` runs in background

Done when: SQLite has continuous time-series rows.

Week 2 — Detect & API

- ☐ Detection engine: RAM critical, memory hog, overflow pattern
- ☐ FastAPI: `/metrics`, `/processes`, `/diagnose`
- ☐ Unit tests for each detection rule with sample data

Done when: `POST /diagnose` returns correct JSON when RAM is low.

Week 3 — Dashboard & AI

- ☐ React dashboard with CPU/RAM charts and process table
- ☐ `POST /analyze` endpoint with LLM explanation
- ☐ Demo script that simulates memory pressure
- ☐ README + 60s screen recording

Done when: Full pipeline works — collector → SQLite → detect → AI explain → dashboard.

MVP Success Criteria

1. Collector writes CPU + RAM + process data to SQLite every 5 seconds
2. Detection engine flags low RAM and memory-hog processes
3. FastAPI serves metrics and diagnosis endpoints
4. Dashboard shows live charts and an AI explanation on button click
5. LLM explanation matches the JSON issues (no hallucinated causes)
6. Demo works: run memory pressure script → alert fires → AI explains

---

Demo Scenario

Memory overflow test:

1. Start collector + API + dashboard
2. Run `python scripts/simulate_memory_pressure.py` (allocates ~80% RAM)
3. Dashboard shows RAM chart dropping, alert appears
4. Click **Analyze** → AI explains: low memory, which process to close, suggested fix

---

Future Ideas

Only after MVP ships:

- Disk space and CPU temperature
- 7-day rolling baselines (z-score anomalies)
- "Why is VS Code slow?" — correlate app launch with metrics
- macOS / Linux collectors
- Benchmark vs similar hardware configs

Getting Started

> **Status:** Pre-development

Prerequisites

- Windows 10/11
- Python 3.12+
- Node.js 20+
- Ollama (optional, for local LLM)

Setup (coming soon)

```
git clone https://github.com/<your-username>/local-pc-performance-diagnoser.git
cd local-pc-performance-diagnoser

pip install -r requirements.txt
cd dashboard && npm install && cd ..

# Terminal 1 — collector
python collector/main.py

# Terminal 2 — API
uvicorn api.main:app --reload

# Terminal 3 — dashboard
cd dashboard && npm run dev
```

License

MIT

Author

Radley Le — SWE / Data / AI Engineer